

```

Returns a tuple of the (column, row) that the tile falls into."""
while True: # Keep asking player until they enter a valid move.
    print(f"Player {playerTile}, enter 1 to {BOARD_WIDTH} or QUIT:")
    response = input("> ").upper().strip()

    if response == "QUIT":
        print("Thanks for playing!")
        sys.exit()

    if response not in COLUMN_LABELS:
        print(f"Enter a number from 1 to {BOARD_WIDTH}.")
        continue # Ask player again for their move.

    columnIndex = int(response) - 1 # -1 for 0-based column indexes.

    # If the column is full, ask for a move again:
    if board[(columnIndex, 0)] != EMPTY_SPACE:
        print("That column is full, select another one.")
        continue # Ask player again for their move.

    # Starting from the bottom, find the first empty space.
    for rowIndex in range(BOARD_HEIGHT - 1, -1, -1):
        if board[(columnIndex, rowIndex)] == EMPTY_SPACE:
            return (columnIndex, rowIndex)

def isFull(board):
    """Returns True if the `board` has no empty spaces, otherwise
    returns False."""
    for rowIndex in range(BOARD_HEIGHT):
        for columnIndex in range(BOARD_WIDTH):
            if board[(columnIndex, rowIndex)] == EMPTY_SPACE:
                return False # Found an empty space, so return False.
    return True # All spaces are full.

def isWinner(playerTile, board):
    """Returns True if `playerTile` has four tiles in a row on `board`,
    otherwise returns False."""

    # Go through the entire board, checking for four-in-a-row:
    for columnIndex in range(BOARD_WIDTH - 3):
        for rowIndex in range(BOARD_HEIGHT):
            # Check for four-in-a-row going across to the right:

```